MAP 2	MODELO	20 minutos

Nome: _____ **Curso:** _____ **Nº:** _____

Teste sem consulta. Escreva código Python com a indentação adequada. Pode assumir que os argumentos respeitam as condições indicadas. Se ficar sem espaço vertical numa caixa, pode e deve continuar ao lado.

1 (5 valores). `Percurso` modela o nome de uma rota e uma lista dos seus tempos, em minutos. Na caixa, complete a declaração em falta na `dataclass` e indique, pela ordem apresentada, o que o programa escreve.

```
from dataclasses import dataclass

@dataclass
class Percurso:
    nome: str
    # declaracao em falta

def atualiza(p: Percurso, atraso: int) -> int:
    p.tempos[0] += atraso
    p.tempos.append(p.tempos[-1] - atraso)
    return sum(p.tempos)

p = Percurso("M1", [4, 7])
print(atualiza(p, 3))
print(p.tempos[0] * len(p.tempos))
```

2 (6 valores). Escreva a função `valores_por_tipo`. O argumento `registos` è uma lista cujos elementos são tuplos `(tipo, valor)`. A função deve devolver um dicionário que associa a cada tipo a lista dos seus valores, pela ordem em que ocorrem. Por exemplo, para `[("A", 2), ("B", 5), ("A", 7)]`, devolve `{"A": [2, 7], "B": [5]}`.

```
def valores_por_tipo(registos: list[tuple[str, int]]) -> dict[str, list[int]]:
```

3 (4 valores). Escreva a função `quantos_cabem`, que devolve quantos valores consecutivos do início da lista podem ser somados sem que a soma ultrapasse `limite`. Pode assumir que todos os valores são não negativos. Por exemplo, para `[3, 2, 4]` e `limite = 5`, devolve 2.

```
def quantos_cabem(valores: list[int], limite: int) -> int:
```

4 (5 valores). Considere a sequência seguinte. Em cada linha, escreva na caixa correspondente o valor da expressão indicada no comentário, depois de executar a instrução dessa linha.

```
a = [2, 4, 6]                # a[0] + a[-1]
b = a                         # a is b
b[1] = b[0] + 1               # a[1] + b[2]
a.append(b[1])                # len(b) + b[-1]
```